only common feature in these numbers is that they tend to be large well rounded hexadecimal numbers.

Activating Output

In order to accomplish our objective of activating the LED lights on the Raspberry Pi board we need to send two messages to the GPIO controller. These messages are transmitted using the following commands: mov, lsl and str. For ARM assembly code almost all instructions are written as three letter codes which are mnemonic and hint what the command does. In this case, "mov" is for move, "lsl" logical shift left, "str" is store register and the one we used in an earlier section, "ldr" is for load register. The lines of code that need to be added to our ongoing assembler code file are as follows:

```
mov r1,#1
lsl r1,#18
str r1,[r0,#4]
```

These commands are used to activate an output to the 16th pin of the GPIO which is wired to the OK/ACT LED. The two main instruction in the code are to get a value into the r1 register. We could use the "ldr" command as we did before but as a matter of programming habit it is wise to deduce the value from a formula rather than enter it directly. To activate the LED pin we need to get the right value in the r1 registry. The first line is a "mov" command and places the number 1 to the base 10 into the r1 registry. The "mov" command is faster than the "ldr" command since it doesn't involve a memory interaction which is what the loading "ldr" command does. But keep in mind that the "mov" command can only be used to load certain values.

Every component on the Pi is labled.

The second line of code uses the "lsl" or logical shift left command. This instructs the assembler to shift the binary value representation for the first instruction left by the second instruc-